

코테 자료구조 정리

1. 개수 세면서 정렬하는 문제 — HashMap vs TreeMap

HashMap 버전

- 먼저 HashMap으로 개수를 센 뒤, 키 리스트를 뽑아서 Collections.sort()로 정렬
- 개수 세기: $O(n)$, 정렬: $O(k \log k)$ (k = 고유 키 수)
- 총 시간복잡도: $O(n + k \log k)$

TreeMap 버전

- TreeMap에 넣으면서 바로 정렬 상태 유지
- 삽입 한 번당 $O(\log k)$, n 번 반복하면 $O(n \log k)$
- 별도 정렬 단계 불필요, 코드가 더 깔끔

어떤 걸 쓸까?

- 정렬 기준이 **키(key) 순서**일 때 → TreeMap이 편함
- 정렬 기준이 **값(value) 순서**일 때 → TreeMap 이점 없음. HashMap으로 세고 entrySet 꺼내서 정렬
- 단순히 "가장 많이 나온 것"만 구할 때 → HashMap + 순회가 $O(n)$ 으로 더 빠름

TreeMap 내부 구조

- Red-Black Tree (자가 균형 이진 탐색 트리) 기반
- 정렬 기준: 키의 자연 순서(Comparable). 숫자 → 오름차순, 문자열 → 사전순
- 커스텀: `new TreeMap<>(Comparator.reverseOrder())`
- put / get / remove 모두 $O(\log n)$

2. 트리 자료구조 총정리

2-1. 기본 용어

용어	설명
루트(Root)	최상단 노드
리프(Leaf)	자식 없는 노드
부모 / 자식	연결 관계
형제(Sibling)	같은 부모의 자식들
깊이(Depth)	루트에서 해당 노드까지 거리
높이(Height)	해당 노드에서 리프까지 가장 먼 거리
차수(Degree)	자식 수

2-2. 이진 트리 (Binary Tree)

자식이 최대 2개인 트리. 여기서 여러 종류가 파생됨.

완전 이진 트리 (Complete Binary Tree)

- 마지막 레벨 빼고 모든 레벨이 꽉 차 있고, 마지막 레벨은 왼쪽부터 순서대로 채움
- 핵심: 이전 레벨이 다 차기 전에는 다음 레벨로 넘어가지 않음 → 한 레벨씩 채우기
- 배열로 표현 가능: 인덱스 i 의 부모 = $(i-1)/2$, 왼쪽 자식 = $2i+1$, 오른쪽 자식 = $2i+2$

포화 이진 트리 (Perfect Binary Tree)

- 모든 레벨이 완전히 꽉 찬 트리

이진 탐색 트리 (BST, Binary Search Tree)

- 왼쪽 < 부모 < 오른쪽 규칙
- 탐색: 평균 $O(\log n)$, 최악 $O(n)$ — 한쪽으로 쏠리면 연결 리스트가 됨
- 중위 순회하면 정렬된 순서로 출력됨

2-3. 자가 균형 이진 탐색 트리

BST의 쏠림 문제를 해결하기 위해 균형을 강제하는 트리들.

Red-Black Tree

- 노드에 빨강/검정 색을 칠하고 규칙 강제
- 핵심 규칙: 루트에서 모든 리프까지 경로에 검정 노드 수가 동일
- 이 제약 때문에 가장 긴 경로가 가장 짧은 경로의 2배를 넘을 수 없음
- 높이가 항상 $O(\log n)$ 으로 보장됨
- 삽입/탐색/삭제 모두 $O(\log n)$
- Java에서: TreeMap, TreeSet의 내부 구현

AVL Tree

- Red-Black보다 더 엄격한 균형 (좌우 서브트리 높이 차이가 최대 1)
- 조화가 많은 경우 유리, 삽입/삭제는 회전이 더 자주 발생

면접 표현: "Red-Black Tree는 자가 균형 이진 탐색 트리라서 탐색이 $O(\log n)$ "

- "이진 탐색이라 $O(\log n)$ " ← 배열의 이진 탐색과 혼동될 수 있으므로 주의
- "자가 균형"을 꼭 붙일 것 — 일반 BST는 최악 $O(n)$ 이니까

2-4. 힙 (Heap)

- 완전 이진 트리 + 부모가 자식보다 항상 크거나(최대 힙) 작음(최소 힙) 구조
- 전체 정렬이 아님! "루트가 항상 최솟값/최댓값"만 보장
- 배열 기반 (완전 이진 트리이므로 배열 인덱스로 부모/자식 접근 가능)
- Java에서: PriorityQueue

시간복잡도

연산	복잡도	동작 방식
offer (삽입)	$O(\log n)$	배열 끝에 넣고 부모와 비교하며 위로 올림 (sift up)
poll (최솟값 제거)	$O(\log n)$	루트 빼고, 배열 마지막 원소를 루트에 놓고 아래로 내림 (sift down)
peek (최솟값 조회)	$O(1)$	루트(배열 인덱스 0)만 보면 됨

poll할 때 "배열 마지막 원소"란?

- 완전 이진 트리에서 가장 밑 레벨의 가장 오른쪽 노드 = 배열의 마지막 인덱스
- 이걸 루트로 옮기는 이유: 완전 이진 트리 형태를 유지하기 위해

일반 큐 vs PriorityQueue

- 일반 큐: 뒤에서 넣고 앞에서 빼는 한 줄 통로 (선입선출)
- PriorityQueue: "대기실" 개념. 어디서 넣든 상관없이, 뺄 때 항상 우선순위 가장 높은 것이 나옴
- 넣는 곳/빼는 곳이 정해진 게 아니라 힙 구조가 최솟값을 루트에 유지

최대 힙: `new PriorityQueue<>(Comparator.reverseOrder())`

2-5. B-Tree / B+Tree

둘 다 다진 트리(m-ary tree) — 이진 트리가 아님. 자식이 여러 개.

B-Tree

- 자가 균형 다진 트리
- 노드 하나에 키가 여러 개
- 모든 노드에 데이터 저장
- 노드가 꽉 차면 분할(split), 너무 비면 병합(merge)

B+Tree

- B-Tree의 변형
- 리프 노드에만 데이터 저장
- 리프끼리 링크드 리스트로 연결 → 범위 검색에 유리
- DB 인덱스의 표준

왜 DB는 B+Tree를 쓰나?

- 목적: 디스크 I/O 최소화
- 노드 하나 = 디스크 블록(4KB~16KB) 하나에 대응
- 한 번 읽으면 키를 수십~수백 개 가져올 수 있음
- 팬아웃이 커서 높이가 3~4 정도로 극도로 낮음 (1억 건이어도 디스크 읽기 3~4번)
- B-Tree도 "한 번에 많이 가져오기"는 같지만, 범위 검색 시 트리를 이리저리 타야 함
- B+Tree는 리프까지 한 번 내려간 후 링크드 리스트를 쭉 따라가면 끝

메모리 vs 디스크 — 자료구조 선택 기준

환경	자료구조	이유
메모리 (TreeMap)	Red-Black Tree	접근 비용이 싸서 포인터 여러 번 따라가도 OK. B+Tree는 오버엔지니어링

환경	자료구조	이유
디스크 (DB 인덱스)	B+Tree	접근 비용이 비싸서 한 번에 많이 가져와야 함. 납작한 구조 + 범위 검색

2-6. 트라이 (Trie)

- 문자열을 한 글자씩 트리 경로로 저장하는 구조
- 접두사가 같은 문자열은 경로를 공유
- 탐색: $O(m)$ — m 은 문자열 길이
- 용도: 자동완성, 사전 검색, 접두사 기반 검색
- 코테 빈도: Lv.3~4 이상 고난도에서 가끔

2-7. 트리 순회

순회 방식	순서	탐색 유형
전위 (Preorder)	부모 → 왼쪽 → 오른쪽	DFS
중위 (Inorder)	왼쪽 → 부모 → 오른쪽	DFS
후위 (Postorder)	왼쪽 → 오른쪽 → 부모	DFS
레벨 (Level-order)	층별로 왼쪽부터	BFS

- BST를 중위 순회하면 정렬된 순서로 나옴
- 코테에서 트리 문제 나오면 거의 반드시 사용

3. Queue / Deque / PriorityQueue와 ArrayDeque

3-1. 구현체 선택: ArrayDeque가 베스트

스택, 큐, 덱(Deque) 전부 **ArrayDeque**로 통일.

ArrayDeque vs LinkedList

비교 항목	ArrayDeque	LinkedList
내부 구조	배열 기반 원형 큐	노드 + prev/next 포인터
시간복잡도	offer/poll/peek 모두 $O(1)$	offer/poll/peek 모두 $O(1)$
실제 성능	빠름 (캐시 지역성 좋음)	느림 (캐시 미스 잦음)
메모리	적음 (배열 하나)	큼 (노드당 메모리 오버헤드)

빅오는 같지만 **노드당 메모리 오버헤드**와 **캐시 지역성(cache locality)** 차이로 실측 성능에서 ArrayDeque가 이김.

3-2. 원형 큐 (Circular Queue) — ArrayDeque의 내부 원리

일반 배열의 문제: 앞에서 빼면 나머지를 한 칸씩 당겨야 해서 $O(n)$. 또는 안 당기면 앞쪽 공간 낭비.

원형 큐의 해결: 데이터는 그대로 두고, head/tail 포인터만 이동.

- 인덱스 이동: $(index + 1) \% capacity$ — 배열 끝에 도달하면 인덱스 0으로 돌아감
- 물리적 배열은 섞여 보이지만 head/tail이 논리적 순서를 관리
- 짝 차면 보통 2배 크기로 리사이즈 (분할 상환 $O(1)$)

핵심 발상: 물리적으로 데이터를 옮기는 게 아니라, 역으로 논리적 포인터를 옮김 $\rightarrow O(1)$

예시 (capacity=5):

```
초기: [1][2][3][4][ ] head=0, tail=4
offer(5): [1][2][3][4][5] head=0, tail=0 ← (4+1)%5
poll(): [ ][2][3][4][5] head=1, tail=0
offer(6): [6][2][3][4][5] head=1, tail=1
→ 물리적으로 섞여 보이지만 head=1부터 읽으면 2→3→4→5→6
```

시간복잡도

연산	복잡도	설명
offer	$O(1)$	tail 위치에 넣고 tail 이동
poll	$O(1)$	head 위치에서 빼고 head 이동
peek	$O(1)$	head 위치 조회
리사이즈	$O(n)$	가끔 발생, 분할 상환(amortized) $O(1)$

3-3. 인터페이스별 사용 가능 메서드

선언하는 인터페이스 타입에 따라 보이는 메서드가 달라짐. 내부 구현은 전부 같은 ArrayDeque.

Queue<> q = new ArrayDeque<>() → 큐 메서드만 **Deque<> dq = new ArrayDeque<>()** → 큐 + 덱 + 스택 메서드 전부 (코테 추천)

```
앞 ===== 뒤

넣기: offerFirst()          offerLast() / offer()
빼기: pollFirst() / poll()  pollLast()
보기: peekFirst() / peek()  peekLast()
```

- 큐로 쓸 때: 뒤에서 넣고(offer) 앞에서 빼기(poll)
- 스택으로 쓸 때: push(= offerFirst) + pop(= pollFirst) — 앞에서 넣고 앞에서 빼기
- offer/poll/peek 시리즈 사용 (실패 시 null/false 반환). add/remove는 예외를 던지므로 비추.

3-4. 괄호 문제 — 스택 vs cnt 변수

- 괄호 종류가 1개(()) → cnt++ / cnt--로 충분. 코테에서는 이게 더 나음
- 괄호 종류가 여러 개([] { }) → 스택 필요. 마지막에 연 괄호가 뭐였는지 확인해야 하니까

부록: 코테 실전 팁

시간복잡도 기준 (1초 $\approx 10^8$ 연산)

입력 크기 n	사용 가능한 복잡도
≤ 10	$O(n!)$
$\leq 20 \sim 25$	$O(2^n)$
≤ 500	$O(n^3)$
$\leq 10,000$	$O(n^2)$
$\leq 100,000$	$O(n \log n)$
$\leq 1,000,000+$	$O(n)$ 또는 $O(\log n)$

→ 문제 읽고 입력 크기 확인 → 역으로 알고리즘 복잡도 상한 추론

int vs long 판단

- 문제 조건에서 값의 범위가 10^9 초과 → `Long.parseLong()` 사용
- 특히 String으로 숫자가 들어올 때 주의 (자릿수가 큰 경우)
- 곱셈이 있으면 `(long) a * b`로 캐스팅
- 배열 인덱스는 int만 받으므로 전부 long으로 통일하는 건 비추